APACHE AIRFLOW — ARCHITECTURE, OPERATIONS & REFERENCE V1

Orchestrate, don't compute — the data interval model, concurrency, and what Airflow 3 removed

concepts · scheduling · authoring · executors · deployment · concurrency · operations · security · **depth marked** junior → staff

1 · WHAT AIRFLOW IS, WHAT IT IS NOT, AND WHAT AIRFLOW 3 CHANGED

start here — the single most valuable principle is that Airflow orchestrates work rather than performing it

The definition

A platform to **author, schedule and monitor workflows as code**. Workflows are Python files defining a **directed acyclic graph** of tasks with dependencies between them. Airflow decides *what runs when*, in what order, and what to do when something fails.

The principle that prevents most Airflow problems: orchestrate, do not compute. Airflow should **tell other systems to do work** — trigger a Spark job, run a warehouse query, call an API — and track the outcome. It should not load a dataframe, join it and write it back. Tasks that process data themselves make workers the bottleneck, make memory a scheduling concern, and make the platform impossible to scale independently of the work.

What it is not, stated plainly

Not a data processing engine	Use Spark, dbt or the warehouse. Airflow launches them
Not a streaming platform	It is batch-oriented. Sub-minute or event-per-record work belongs in Kafka, Flink or a queue consumer
Not a low-latency scheduler	Scheduling latency is measured in seconds. It is not a cron replacement for second-level precision
Not an ETL tool	It is the conductor. The instruments are elsewhere

Where it genuinely excels: batch pipelines with real dependency graphs, heterogeneous systems that must be coordinated, work needing retries and backfills, and anywhere the **schedule and dependency logic itself is complex enough to deserve version control.**

STAFF **Airflow 3 arrived in 2025 and it is a genuine break, not a point release.** A reference written for 2.x will teach several things that no longer work. The headline changes are worth knowing whichever version you run, because they signal the direction.

What Airflow 3 added

- **Task SDK and a Task Execution API** — tasks run with **full isolation from the metadata database**, which removes a long-standing security and coupling problem. Authors now import from the `airflow.sdk` namespace
- **DAG versioning** — structural changes are tracked over time, so you can inspect the DAG as it was when a historical run executed. This closes a real gap in debugging
- **Assets** — the dataset model renamed and redesigned, with an `@asset` decorator and a cleaner event-driven model. Airflow can now **watch external assets and trigger on change**
- **Edge Executor** for distributed and edge workloads, now generally available
- **A rebuilt UI** on React and FastAPI, with version-aware views and backfill management
- **Scheduler-managed backfills**, rather than a separate CLI-driven mechanism

What Airflow 3 removed — check your DAGs against this list. SubDAGs and `SubDagOperator` are gone entirely; use TaskGroups. **SequentialExecutor** is gone; LocalExecutor now handles SQLite and is the local-testing answer. **SLAs** are removed. **DAG and XCom pickling** are removed. `execution_date` is removed. And `schedule_interval`, deprecated since 2.4, is removed in favour of `schedule`.

Deprecated patterns still widely copied

Do not use	Use instead
`from airflow.utils.dates import days_ago`	**Removed.** Use a fixed date, or `pendulum.today("UTC").add(days=-N)`. A dynamic `start_date` was always a bug — it makes the schedule shift every time the file is parsed
`schedule_interval=`	`schedule=` — deprecated in 2.4, removed in 3
`execution_date`	`logical_date`, and the data interval fields. See panel 2
`SubDagOperator`	`TaskGroup`
`SequentialExecutor`	`LocalExecutor`
`from airflow import DAG`	On 3.x, `from airflow.sdk import DAG, dag, task`

A note on the ecosystem. Airflow's real strength is the **provider packages** — hundreds of them, covering every major cloud, database, warehouse and SaaS platform. Before writing a custom operator, check whether a provider already does it; a maintained provider handles authentication, pagination, retries and deferrability that a hand-written hook will not.

2 · CORE CONCEPTS AND THE DATA INTERVAL — THE PART EVERYONE GETS WRONG

JUNIOR the vocabulary · STAFF why your daily DAG runs a day "late", and why that is correct

Concept	What it is
DAG	A Python file describing tasks and their dependencies. **A definition, not a run**
Task	A node in the graph — one unit of work
Operator	A template for a task. `BashOperator`, `PythonOperator`, and hundreds from providers
Task instance	One specific execution of a task, for one DAG run. **This is what actually has a state**
DAG run	One execution of the whole DAG, for one logical date
Hook	The connection layer to an external system. Operators use hooks
Sensor	A task that waits for a condition. **See panel 11 — these cause more trouble than any other task type**
XCom	Small messages passed between tasks, stored in the metadata database
Pool	A named concurrency limit shared across tasks
Asset	A logical dataset a DAG produces or consumes, enabling **data-aware scheduling** instead of time-based

The state model to know: a task is `scheduled` → `queued` → `running` → `success` or `failed`, with `up_for_retry`, `up_for_reschedule`, `skipped` and `upstream_failed` along the way. **Tasks stuck in queued mean the executor cannot place them** — a capacity or pool problem, not a code one.

The data interval is the single most confusing concept in Airflow, and it is worth being precise about. A scheduled DAG run does not represent "now". It represents a **period of data** — and it runs **after that period has ended**, because only then is the data complete.

```
# a DAG with schedule="@daily"

data_interval_start   2026-03-01 00:00
data_interval_end     2026-03-02 00:00
                            ↓
the run is triggered at 2026-03-02 00:00
# because only then is 1 March complete

# so on the morning of 2 March, the most recent
# successful run is labelled 1 March.
# This is correct. It is not a delay.
```

Use the interval, not the clock. A task that queries WHERE `date = today()` is not idempotent and will produce different results on a rerun or a backfill. Use `data_interval_start` and `data_interval_end` from the context, and the same task run at any time produces the same result — **which is what makes backfills meaningful.**

A version note that matters when reading older material. In Airflow 2, `logical_date` equalled `data_interval_start`. **In Airflow 3 it equals the run-after time** instead. The data interval fields are unchanged and are what you should reference in task logic either way.

And the footgun: `catchup` defaults to true. Deploy a DAG with a `start_date` two years ago and Airflow will attempt to run **every missed interval** — hundreds of runs launching at once, saturating workers and possibly hammering a source system. **Set `catchup=False` unless you specifically want history**, and pair it with `max_active_runs`.

3 · SCHEDULING

time-based and data-aware

Form	Example
Cron	`"0 6 * * *"` — daily at 06:00. Standard five-field syntax
Presets	`@daily`, `@hourly`, `@weekly`, `@monthly`, `@once`
Timedelta	`timedelta(hours=6)` — relative to the previous run rather than to a wall clock
None	Triggered manually or by another DAG only
Timetable	A custom Python class. **The answer for business days, trading calendars or irregular schedules** that cron cannot express
Assets	Run when an upstream DAG produces a dataset — **data-aware rather than time-based**

Asset-based scheduling is the most under-used feature here. Instead of guessing that upstream finishes by 06:00 and scheduling for 06:30, declare that your DAG consumes an asset the upstream DAG produces. It then runs **when the data is actually ready** — no padding, no silent breakage when upstream slows down. Airflow 3 extends this to watching external assets and triggering on external events.

Catchup and backfill

- **`catchup=True` is the default** and will run every interval since `start_date`
- **Backfill** re-runs a historical range deliberately. In Airflow 3 this is scheduler-managed and visible in the UI rather than a detached CLI process
- **Bound it.** A backfill over two years of daily partitions launched at full concurrency will saturate the cluster — use `max_active_runs` and a pool
- **Backfill only works if tasks are idempotent** and derive their window from the data interval

Trigger rules

`all_success`	**The default.** Run when every upstream succeeded
`all_done`	Run when upstreams finish, whatever the outcome. **The one to use for cleanup and notification tasks**
`one_success` `one_failed`	Run as soon as any upstream succeeds or fails
`none_failed_min_one_success`	The correct rule for a task that joins branches
`always`	Run regardless

Branching needs the right join rule. After a branch, the untaken path is `skipped` — and the default `all_success` treats a skipped upstream as not satisfied, so the join never runs. `none_failed_min_one_success` is what you want.

4 · ARCHITECTURE — THE COMPONENTS, AND HOW A TASK ACTUALLY GETS RUN

what each process does, where state lives, and the loop that turns a DAG file into a running task

THE COMPONENTS — each is a separate process, and each scales independently

SCHEDULER — the heart of it
· Reads parsed DAGs and decides what is due
· Creates DAG runs and task instances
· Checks dependencies, pools and concurrency
· Hands ready tasks to the executor
Run two or more — HA is supported since 2.0

DAG PROCESSOR
· Parses Python files into DAG objects
· Runs repeatedly on an interval, *per file*
· Writes serialised DAGs to the metadata DB
Slow parsing here delays every schedule. This is where top-level code hurts you.

EXECUTOR
· Decides *where* a task runs
· Local, Celery, Kubernetes or Edge
· Runs inside the scheduler process
Not a separate service — a strategy the scheduler uses. See panel 6.

WORKERS
· Where task code actually executes
· Celery workers, or Kubernetes pods
· Need the DAG files and all dependencies
Scale these for throughput. Everything else is coordination.

TRIGGERER
· Runs deferrable operators asynchronously
· One process handles thousands of waits
· Frees worker slots while waiting
Deploy it. Without it, deferrable operators simply do not work — and sensors eat slots.

API SERVER / WEBSERVER
· UI, REST API, logs, DAG and run inspection
· In Airflow 3, also serves the Task Execution API
· Stateless — run several behind a load balancer
Losing it stops visibility, not execution. Scheduled tasks keep running without it.

METADATA DATABASE — every component's shared state, and the single point of failure worth protecting

PostgreSQL or MySQL 8. It holds DAG runs, task instance states, serialised DAGs, XComs, connections, variables, pools, users and every log reference. **If it is down, Airflow is down.**

Every scheduler loop, every UI page load and — in Airflow 2 — every task queries it. Run it as a managed, highly available instance with real monitoring, not as a container beside the scheduler.

It also grows without limit. Task instance, log and XCom tables accumulate forever unless you run a cleanup job — see panel 12. An unmaintained metadata database is the most common cause of a gradually slowing Airflow, and the symptom looks like "the scheduler got slow" rather than like a database problem.

HOW A TASK GETS RUN — the loop

1 · The DAG processor parses the Python file and writes the serialised DAG to the metadata database.
2 · The scheduler reads it, sees an interval has completed, and creates a **DAG run** plus a **task instance** per task.
3 · Each loop it checks which task instances have satisfied dependencies, and whether pools and concurrency limits allow them.
4 · Eligible tasks move to `queued` and are handed to the executor.
5 · The executor places them — a local subprocess, a Celery queue, or a new Kubernetes pod.
6 · The worker runs the task and reports state back.

A task stuck in queued is step 5 failing — no capacity, a full pool, or workers that cannot start. It is never a problem with your code.

READING THE DIAGRAM — five properties that explain the rest of this sheet

1 · DAG files are parsed repeatedly, not once. Every file is re-executed on an interval — commonly every thirty seconds. Any code at module level runs every single time: an API call, a database query, a `Variable.get()`. This is the single largest performance mistake available in Airflow, and it presents as a slow scheduler rather than as a slow DAG. Keep module level to imports and cheap literals; put everything else inside tasks.

2 · The metadata database is the shared state and the hard dependency. It is queried constantly by every component. Make it highly available, size it properly, and clean it on a schedule — it grows forever by default, and a bloated instance slows every scheduler loop and every UI page. Airflow 3's Task Execution API removes direct database access from tasks, which is a meaningful reduction in both load and blast radius.

3 · Workers are where throughput lives; everything else is coordination. If tasks are slow, add workers or make the task do less. If tasks are not *starting*, the problem is the scheduler, a pool, or a concurrency limit — a completely different investigation, and the distinction between the two is the fastest triage available.

4 · Retries are automatic, so every task must be idempotent. A task that appends rather than replacing will duplicate data the first time it retries — and it will retry, on some night nobody is watching. Derive the window from the data interval and overwrite or merge rather than appending. This is not a best practice; it is a requirement of the execution model.

5 · Losing the webserver is not an outage. Scheduling and execution continue without it — you lose visibility and manual control, not the pipelines. Losing the scheduler stops new work; losing the metadata database stops everything.

Useful interrogation of any Airflow deployment: how long does DAG parsing take, is the metadata database cleaned on a schedule, what are the concurrency limits at each of the four levels, is a triggerer deployed, and how do DAG files reach the workers?

Those five questions surface most production problems — and the first and last are the ones teams most often cannot answer.

5 · WRITING DAGS — THE MODERN PATTERNS

MID TaskFlow and dynamic mapping · STAFF the top-level code rule

```
# Airflow 3 — the airflow.sdk namespace
from airflow.sdk import dag, task
import pendulum

@dag(
    # a FIXED start date, never a relative one
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    schedule="0 6 * * *",       # not schedule_interval
    catchup=False,              # default is True!
    max_active_runs=1,
    default_args={"retries": 3,
                  "retry_delay": timedelta(minutes=5)},
    tags=["example"],
)
def daily_pipeline():

    @task
    def extract(**context):
        # use the interval, never datetime.now()
        start = context["data_interval_start"]
        end   = context["data_interval_end"]
        return fetch_rows(start, end)

    @task
    def load(rows):  # dependency is implicit
        write(rows)

    load(extract())

daily_pipeline()
```

The TaskFlow API is the modern way to write DAGs. Decorated functions, dependencies inferred from how values are passed, and XComs handled for you. It is far more readable than explicitly wiring operators with >>, and it is what new material assumes.

The rule that matters most: nothing expensive at module level. The DAG file is re-executed on every parse — **commonly every thirty seconds, forever.** A `Variable.get()`, a database query, an API call or a large import at module level runs every time, for every file. It slows every schedule for every DAG, and it presents as "the scheduler is slow" rather than as a problem with your file.

```
# WRONG — runs every 30 seconds, forever
config = Variable.get("my_config")
df = pd.read_csv("s3://bucket/big.csv")

# RIGHT — runs only when the task runs
@task
def work():
    config = Variable.get("my_config")
```

Dynamic task mapping

```
@task
def process(f): return handle(f)

# one task instance per file, decided at runtime
process.expand(f=list_files())
```

Creates a variable number of parallel task instances from a runtime value. **This replaced the old pattern of generating tasks in a loop at parse time**, which forced the DAG structure to be known before the run and made parsing expensive.

Structure and organisation

- **TaskGroups** for visual and logical grouping — **SubDAGs are removed**, and TaskGroups were always the better answer
- **One DAG per logical pipeline.** Hundreds of tasks in one DAG is hard to operate and slow to parse
- **Keep tasks small and single-purpose** — a failed task should be re-runnable on its own
- **Put shared logic in an importable module**, not copied across DAG files
- **Set `retries` and `retry_delay`** in `default_args`. The default of no retries is rarely what you want

6 · EXECUTORS

where tasks actually run

Executor	Use, and trade-offs
LocalExecutor	Runs tasks as subprocesses on the scheduler host. **Simple, genuinely useful, and correct for small production deployments.** Bounded by one machine, and now the answer for local development too
CeleryExecutor	**The most common production choice.** Distributes tasks to a pool of persistent workers via Redis or RabbitMQ. Mature and predictable; workers are always running, so you pay for idle capacity, and the broker is another component to operate
KubernetesExecutor	**One pod per task.** Strong isolation, per-task resources and images, and scales to zero. Pod startup adds latency per task, which hurts for many short tasks, and it requires real Kubernetes competence
EdgeExecutor	Added in Airflow 3 for distributed and edge-compute workloads where workers sit outside the core cluster
Multiple executors	Recent versions allow **configuring several and choosing per DAG or per task** — which supersedes the older hybrid executors

Two corrections worth noting. SequentialExecutor has been removed — LocalExecutor handles SQLite and is the local answer. And the **hybrid Celery-Kubernetes executor is superseded** by native multi-executor configuration, which is cleaner and does not require a special executor class.

Choosing, briefly. Small and simple → **LocalExecutor**. Steady load with many similar tasks → **Celery**. Varied resource needs, per-task dependencies, or spiky load where scale-to-zero matters → **Kubernetes**. Already on Kubernetes with a platform team → Kubernetes. Otherwise Celery is the lower operational burden.

The Kubernetes trade-off, precisely. Pod startup is measured in seconds. For tasks that run for minutes or hours, that is irrelevant and the isolation is worth a great deal. For thousands of tasks that each take five seconds, the overhead dominates and Celery will be dramatically better.

7 · DEPLOYMENT — TOPOLOGY, AND GETTING DAG FILES TO THE WORKERS

MID the topologies · STAFF DAG delivery, which is the decision people skip

Topology	Shape
Single node dev and test	Everything on one host with LocalExecutor and Postgres. Fine for development, and **acceptable for small production** if you accept the single point of failure
Celery cluster common production	Two or more schedulers, several API servers behind a load balancer, a triggerer, a worker fleet, a Redis or RabbitMQ broker, and a managed Postgres. **The standard shape**
Kubernetes	The same components as deployments, with tasks as pods. Deployed via the official Helm chart or a vendor distribution
Managed	Astronomer, AWS MWAA, Google Cloud Composer, Azure Data Factory Managed Airflow. **You give up some configuration control and stop operating the metadata database** — usually a good trade

Run more than one scheduler. Highly available active-active schedulers have been supported since Airflow 2.0, they need no leader election, and the second one removes the most obvious single point of failure. There is little reason to run only one in production.

Sizing, as a starting point

Scheduler	2–4 vCPU, 4–8 GB. **CPU-bound on DAG parsing**, so parsing cost drives this more than DAG count
API server	2 vCPU, 4 GB. Scale horizontally for many UI users
Worker	Entirely workload-dependent. **Size from what tasks actually do**, and remember that if tasks only trigger external systems these can be small
Triggerer	Modest — one process handles thousands of concurrent deferrals
Metadata DB	4+ vCPU, 8+ GB, fast disk. **Highly available.** The most important component to get right

The decision most deployment guides omit: how do DAG files reach every component? The scheduler, the DAG processor and **every worker** need the same files and the same Python dependencies. Getting this wrong produces the confusing failure where a DAG is visible in the UI and its tasks fail on the worker with an import error.

Method	Trade-offs
Baked into the image	**The most reproducible.** DAGs and dependencies are versioned together and a deploy is an image roll. Changing a DAG requires a deployment, which is a feature in regulated settings and friction in fast-moving ones
Git-sync sidecar	A sidecar pulls from a repository on an interval. **Fast iteration, no redeploy.** Python dependencies still come from the image, so they can drift out of step with the DAGs
Shared volume	A network filesystem mounted everywhere. Simple, and it introduces a shared dependency and a performance risk on parse
Object storage sync	Used by several managed offerings — you upload to a bucket and it distributes

Whichever you choose, the dependency problem is the same one. A DAG importing a library the worker image lacks fails only at task runtime. **Keep DAG and dependency versions moving together**, and test the image the workers will actually use.

Logs

- **Configure remote logging** to object storage from day one. Local logs disappear when a Kubernetes pod exits, and the UI then shows nothing for the run you most want to inspect
- **Set a retention policy** on the bucket — logs accumulate indefinitely otherwise
- **The metadata database stores log *references***, not log content — but the task instance rows themselves still grow

8 · CONFIGURATION

the settings that change behaviour

Setting	Why it matters
`core.parallelism`	**Cluster-wide cap** on running task instances. The outermost limit
`core.max_active_tasks_per_dag`	Per-DAG cap. Stops one DAG monopolising the cluster
`core.max_active_runs_per_dag`	How many DAG runs may be active at once. **Critical when catchup or a backfill is involved**
`celery.worker_concurrency`	Tasks per Celery worker process. Size against what tasks actually consume
`scheduler.min_file_process_interval`	How often a DAG file is re-parsed. **Raise it if parsing is your bottleneck** — at the cost of slower pickup of edits
`scheduler.dag_dir_list_interval`	How often the folder is scanned for new files
`scheduler.parsing_processes`	Parallel parsing workers. Raise for many DAG files
`core.dagbag_import_timeout`	A file exceeding this is skipped. Silent, and worth alerting on
`core.load_examples`	**Set to False in production.** Example DAGs add parsing cost and clutter
`core.fernet_key`	Encrypts connection passwords at rest. **Set it, and manage it as a secret** — losing it makes stored connections unreadable
`logging.remote_logging`	Ship logs to object storage. Essential on Kubernetes
`secrets.backend`	Fetch connections and variables from a real secrets manager

Configuration precedence, which trips people up: environment variables of the form `AIRFLOW__SECTION__KEY` override `airflow.cfg`, which overrides the defaults. **Environment variables are the right mechanism** in containers, and they keep secrets out of a config file.

Tune parsing before adding hardware. If the scheduler is struggling, measure DAG parse time first — the fix is usually removing top-level code or raising the parse interval, not a bigger machine.

9 · CONCURRENCY & POOLS

four limits, all applying at once

Tasks stuck in queued are almost always hitting one of these, and the confusion comes from there being four independent ceilings. The most restrictive one wins, and the UI does not tell you which.

```
1. parallelism             whole cluster
        ↓
2. max_active_tasks_per_dag  one DAG
        ↓
3. pool slots              a named group
        ↓
4. worker_concurrency      one worker process

# plus max_active_runs_per_dag, which limits
# how many DAG RUNS may be active at once
```

Pools — the tool for protecting other systems

- **A named set of slots** that tasks must acquire before running
- **The correct way to protect a fragile downstream** — a legacy database that tolerates five concurrent connections gets a pool of five, and Airflow will never exceed it however many DAGs want to run
- **Tasks can request more than one slot**, so a heavy task can consume proportionally more
- **`priority_weight`** decides which queued task takes a freed slot first

Pools are the most under-used feature in Airflow. They convert "we overwhelmed the source system again" from a recurring incident into a configuration value — and unlike staggering schedules by hand, they keep working when the DAG count grows.

Queues, with Celery

- **Route tasks to specific worker groups** — a memory-heavy task to large workers, everything else to standard ones
- Set `queue` on the task, and start workers listening to matching queues
- **A queue with no listening worker means tasks queue forever**, silently. Check this first when tasks never start

The diagnostic order for a stuck task: is the pool full, is the DAG at its task limit, is the cluster at `parallelism`, are workers alive and listening to the right queue? **In that order** — it resolves nearly every case quickly.

10 · XCOM, VARIABLES & CONNECTIONS

and their limits

XCom — small messages only

XComs are stored in the metadata database, which sets the rule: **pass references, never payloads.** A path, an identifier, a row count or a status is correct. A dataframe, a file's contents or a large JSON document is not — it bloats the database, slows every scheduler query, and can exceed the column limit outright.

```
# WRONG
return df                 # megabytes into the DB

# RIGHT
df.to_parquet(path)
return path               # a reference
```

- **A custom XCom backend** can transparently store values in object storage and pass a pointer — the right answer when passing larger objects is genuinely needed
- **TaskFlow uses XComs automatically** for return values, so the same size discipline applies to what your functions return
- **XCom pickling was removed** in Airflow 3 — values must be JSON-serialisable, or handled by a custom backend

Variables

- Key-value configuration stored in the database or a secrets backend
- **Never call `Variable.get()` at module level** — it runs on every parse of every file, hammering the database. Inside a task, or via a Jinja template, is correct
- **Use a single JSON variable** for related settings rather than many separate ones, to reduce lookups

Connections

- Credentials and endpoints for external systems, referenced by `conn_id`
- **Encrypted at rest with the Fernet key** — which must be set and safely stored, or existing connections become unreadable
- **Prefer a secrets backend** — AWS Secrets Manager, HashiCorp Vault, GCP Secret Manager. Connections then live where the rest of your secrets live, with rotation and audit
- **Never put credentials in DAG code.** It is version controlled, and it is parsed and logged

11 · SENSORS & DEFERRABLE OPERATORS

the classic resource trap

The classic Airflow outage: sensor deadlock. A sensor in its default mode **occupies a worker slot for its entire wait.** Schedule twenty sensors waiting on files that arrive at midday, with sixteen worker slots, and **every slot is consumed by tasks doing nothing** — including the tasks that would satisfy them. The cluster is fully utilised and completely stuck.

The three approaches, in increasing order of correctness

Mode	Behaviour
poke the default	Holds the worker slot and checks on an interval. **Fine only for short waits** measured in seconds or a few minutes
reschedule	**Releases the slot between checks** and returns to the queue. Far better for long waits; each check still costs a task startup, so a short poke interval is wasteful
Deferrable	**The right answer.** The task suspends entirely and its wait is handled by the **triggerer** as an async coroutine. Occupies **no worker slot at all**, and one triggerer handles thousands of concurrent waits

Prefer deferrable operators for anything that waits — a sensor, a long-running external job, an API poll. Most major providers ship deferrable variants. This is the single highest-value change available to a cluster whose workers are always busy but whose throughput is low.

The triggerer must actually be deployed. If it is not running, deferrable operators defer and never resume — tasks sit indefinitely with no error. Check for a triggerer before debugging anything else when a deferred task appears stuck.

Better still: do not wait at all

- **Asset-based scheduling** — trigger on data being produced rather than polling for it
- **Event-driven triggers** — Airflow 3 can watch external assets, such as a message queue, and start a DAG on the event
- **Always set a timeout** on any sensor. A sensor with no timeout waits forever, and will still be waiting weeks later

12 · OPERATIONS — KEEPING IT HEALTHY OVER YEARS

STAFF metadata database maintenance, parse-time budgets, and the 2 to 3 upgrade

The metadata database grows forever, and nobody notices until it is a problem. Task instance rows, log references, XComs, DAG run records and serialised DAG versions accumulate with every run. After a year of a few hundred daily tasks the tables are large enough that **every scheduler query and every UI page slows down.** It presents as "Airflow got slow".

```
# run this on a schedule, from the start
airflow db clean --clean-before-timestamp <date>

# preview first
airflow db clean --dry-run --clean-before-timestamp <date>
```

Schedule it, keep a retention window you actually need, and take a backup first. Ninety days is a common choice. Pair it with a vacuum or equivalent so the space is genuinely reclaimed — deleting rows in Postgres does not shrink the table on its own.

DAG parse time is a budget

- **Measure it** — `airflow dags report` shows per-file parse duration
- **The usual cause of slowness is top-level code**, not DAG count
- **Raise `min_file_process_interval`** if parsing is the bottleneck and you can accept slower pickup of edits
- **Raise `parsing_processes`** for many files
- **Keep non-DAG Python out of the DAGs folder**, or use `.airflowignore`

Upgrading from Airflow 2 to 3

- **It is a genuine migration, not a version bump.** Budget for it properly
- **Audit for removed features first** — SubDAGs, SequentialExecutor, SLAs, `execution_date`, `schedule_interval`, pickling
- **Move imports to `airflow.sdk`** for DAG constructs
- **Upgrade to the latest 2.x first**, resolve every deprecation warning, then move
- **Test on a copy of the metadata database**, and time the migration — on a large, uncleaned database it can be long
- **Clean the database before migrating.** Less data means a faster and safer migration
- **Check provider compatibility** for every provider you depend on

Routine practices

- **Rolling restarts** — schedulers are active-active, so restart one at a time
- **Watch scheduler heartbeat**. A scheduler that has stopped heartbeating is the most urgent alert here
- **Set log retention** on the remote store
- **Pin provider versions** and upgrade deliberately — providers release independently of core
- **Keep a staging environment** that matches production, including the worker image

Zombie tasks — a task whose worker died without reporting state. The scheduler detects and fails them after a threshold. **Frequent zombies mean workers are being killed**, usually by memory limits, and the real fix is task memory or worker sizing rather than the detection setting.

13 · SECURITY

it runs arbitrary code

Start from what Airflow actually is: a system that executes arbitrary Python with access to every credential your pipelines use. Write access to the DAGs folder is equivalent to code execution with those credentials. Treat it with the seriousness that implies.

The controls

- **Role-based access control** — built in, with roles down to per-DAG permissions. **Never leave authentication disabled**, and never expose the UI publicly
- **Integrate with your identity provider** — LDAP, OAuth, SAML or OIDC — rather than managing local users
- **A secrets backend** so credentials live in Vault or a cloud secrets manager, not in the metadata database
- **Set the Fernet key** and manage it as a secret. It encrypts connection passwords at rest
- **Never put secrets in DAG code.** It is in version control, parsed repeatedly, and may appear in logs
- **Least privilege for the credentials Airflow holds** — a task that reads one table needs read on one table
- **Review DAG folder write access as production code access**, because that is what it is
- **Enable audit logging** and retain it

STAFF **Airflow 3 improved this structurally.** The Task Execution API means **tasks no longer connect directly to the metadata database.** Previously any task could in principle read every connection and variable in the system; now task execution is isolated behind an API, which is a real reduction in blast radius and the main security argument for upgrading.

Multi-tenancy remains limited. RBAC restricts what users see and trigger in the UI, but tasks from different teams share workers and their credentials. **For genuine isolation between untrusted teams, run separate deployments** — this is what managed platforms do, and it is the honest answer.

14 · MONITORING, TESTING AND CI/CD

what to watch, how to test a DAG before it reaches production, and what a pipeline for pipelines looks like

What to monitor

Signal	Why
Scheduler heartbeat	**The most urgent alert.** A scheduler that stops heartbeating means nothing new is being scheduled — and DAGs simply stop, quietly
DAG parse duration	Rising parse time delays every schedule. The leading indicator of scheduler trouble
Task queue depth and time in queued	Tasks waiting means insufficient capacity or a limit being hit. **Distinguishes a capacity problem from a slow-task problem**
Task duration trends	A task that has grown from four minutes to fifty is a warning before it is an incident
Failure and retry rates	Per DAG. Rising retries indicate an unstable dependency
DAG run completion time	Against the freshness the business expects
Pool utilisation	A permanently full pool is a bottleneck by design — verify it is the intended one
Worker health and memory	Killed workers produce zombie tasks
Metadata DB size and query time	Growth predicts the slow-Airflow problem months ahead

Airflow emits metrics via StatsD, and increasingly via OpenTelemetry — feed them to Prometheus, Datadog or equivalent. **The built-in UI is for inspection, not alerting**; a dashboard nobody is watching at 03:00 is not monitoring.

Alerting on the right thing

- **Alert on a dataset being late, not only on a task failing.** A DAG that never started produces no failure and no alert — which is the more dangerous case
- **Use callbacks** — `on_failure_callback` and `on_success_callback` — to route to your paging system rather than email alone
- **Route per DAG to the owning team**, not to a shared inbox nobody reads
- **Note that SLAs were removed in Airflow 3** — implement freshness expectations with external monitoring or callbacks instead
- **Suppress alert storms.** One upstream failure can fail fifty downstream tasks; alert on the root, not on every consequence

Testing DAGs

- **Import test in CI** — parse every DAG and assert there are no import errors and no cycles. **This alone catches most broken deploys** and takes minutes to set up
- **Assert structure** — expected task count and dependency relationships
- **Unit test the logic**, which should live in importable functions rather than inside operators
- **`airflow dags test`** runs a DAG for a logical date locally, end to end
- **`airflow tasks test`** runs a single task without recording state — the fastest inner loop
- **Check parse time in CI** and fail the build if a DAG exceeds a threshold

CI/CD for pipelines

1. **DAGs in version control**, reviewed like any other code
2. **Lint and import-test** on every pull request
3. **Build the worker image** with pinned dependencies, and test against it
4. **Deploy to staging** that matches production, including the image
5. **Promote deliberately** — image roll or git-sync from a release branch
6. **Keep DAG and dependency versions together**, so a DAG never lands ahead of the library it needs

The import test is the highest return on effort available. A DAG with a syntax or import error does not merely fail — it may not appear at all, and the scheduler carries on quietly without it. Catching that in CI prevents the failure mode where a pipeline has silently not run for a week.

Useful CLI

```
airflow dags list
airflow dags report          # parse times
airflow dags test <id> <date>
airflow tasks test <dag> <task> <date>
airflow dags backfill ...
airflow db clean --clean-before-timestamp ...
airflow config list          # effective config
airflow providers list
```

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

what actually goes wrong, and the order in which you will meet it

Symptom	Cause	Fix
Tasks stuck in queued, never start	A concurrency limit, a full pool, no worker on that queue, or workers down	Check in order: pool, per-DAG limit, `parallelism`, worker queue subscription
Everything scheduled late	DAG parsing is slow — usually top-level code	`airflow dags report`; move work into tasks; raise the parse interval
Hundreds of runs launched on deploy	`catchup` defaults to true with an old `start_date`	`catchup=False` and `max_active_runs`
Workers busy, throughput near zero	Sensors in poke mode holding every slot	Deferrable operators, or `reschedule` mode. Deploy the triggerer
Airflow slower every month	Metadata database growing without cleanup	Schedule `airflow db clean`; vacuum; set a retention window
DAG visible in UI, task fails on worker	The worker image lacks a dependency the DAG imports	Version DAGs and dependencies together; test against the worker image
Duplicate rows after a retry	A non-idempotent task that appends	Derive the window from the data interval; overwrite or merge
DAG silently missing from the UI	Import error, or parse timeout — the scheduler skips it	**Import-test in CI.** Alert on import errors and on parse timeouts
Deferred task never resumes	No triggerer running	Deploy a triggerer. Check this before anything else
Scheduler OOM or metadata DB overloaded	Large XComs, or `Variable.get()` at module level	Pass references not payloads; move variable lookups into tasks
Zombie tasks	Workers killed, usually by memory limits	Right-size workers; reduce per-task memory; do the heavy work elsewhere

What breaks first

#	Limit
1	**DAG parsing time**, from top-level code. The most common cause of a slow Airflow, and rarely the first thing suspected
2	**Concurrency limits** — four ceilings, and the UI does not say which one you hit
3	**Sensors consuming worker slots**, producing full utilisation with no throughput
4	**Metadata database growth** — gradual, then everything is slow
5	**Raw task throughput** — the thing people size for, and the last to bind

Note the order. None of the top four are solved by adding workers. They are parse hygiene, configuration, task design and maintenance — all visible in advance if anyone is measuring them.

Degradation

- **Webserver down** — you lose visibility and manual control. **Scheduling and execution continue.** Not an emergency
- **One scheduler down** — no impact if you run two. This is the argument for running two
- **All schedulers down** — nothing new is scheduled; running tasks finish. **Silent, so alert on the heartbeat**
- **Workers down** — tasks queue and resume when capacity returns, provided the queue and broker survive
- **Metadata database down** — **total outage.** Every component depends on it, which is why it must be highly available

16 · PRODUCTION CHECKLIST

before pipelines depend on it

- ☐ **Airflow orchestrates, does not process data**
- ☐ **No expensive code at module level** — no API calls, no `Variable.get()`
- ☐ **DAG parse time measured** and checked in CI
- ☐ **`start_date` is static** — never `days_ago` or a relative value
- ☐ **`catchup=False` unless history is wanted**
- ☐ **`max_active_runs` set** on every DAG
- ☐ **Tasks use the data interval**, never `datetime.now()`
- ☐ **Every task idempotent** — overwrite or merge, never append
- ☐ **Retries and `retry_delay` set** in `default_args`
- ☐ **Correct trigger rule on branch joins**
- ☐ **No deprecated constructs** — `schedule_interval`, SubDAGs, `execution_date`
- ☐ **Imports from `airflow.sdk`** on Airflow 3
- ☐ **TaskGroups instead of SubDAGs**
- ☐ **Dynamic task mapping** instead of parse-time loops
- ☐ **XComs carry references, not payloads**
- ☐ **Custom XCom backend** if larger objects must be passed
- ☐ **Sensors are deferrable** or use `reschedule` mode
- ☐ **Every sensor has a timeout**
- ☐ **Triggerer deployed** and monitored
- ☐ **Asset-based scheduling used** where it replaces polling
- ☐ **Two or more schedulers running**
- ☐ **Executor chosen deliberately** for the workload shape
- ☐ **Metadata DB is managed and highly available**
- ☐ **`airflow db clean` scheduled** with a retention window
- ☐ **Remote logging configured** with bucket retention
- ☐ **DAG delivery method chosen**; dependencies versioned with DAGs
- ☐ **Worker image tested** with the DAGs it will run
- ☐ **All four concurrency limits set deliberately**
- ☐ **Pools protecting fragile downstream systems**
- ☐ **Queues have listening workers**
- ☐ **`load_examples` disabled** in production
- ☐ **Fernet key set** and stored as a secret
- ☐ **Secrets backend configured**; no credentials in DAG code
- ☐ **Authentication enabled**; UI not publicly exposed
- ☐ **RBAC roles per team**; DAG folder write access treated as code access
- ☐ **Scheduler heartbeat alerted on** — the most urgent signal
- ☐ **Alerting on data being late**, not only on task failure
- ☐ **Alerts route to owning teams**; storms suppressed
- ☐ **Import test in CI** for every DAG
- ☐ **Staging environment matches production**, including the image

The one that summarises the sheet: **state how long DAG parsing takes, whether the metadata database is cleaned on a schedule, what the four concurrency limits are set to, whether a triggerer is running, and how DAG files and their dependencies reach the workers.** Those five answers cover the failures that actually take Airflow down — and not one of them is about how many tasks per second it can run.